1.)

-Deep Learning for Automated Inspection and Vision

Deep learning, especially Convolutional Neural Networks (CNNs), has more become important for inspection tasks in manufacturing. These models can be automatically learn features from images, which makes them useful for detecting defects and identifying objects even when the contrast is low or the surface conditions vary. Compared to traditional image processing, CNNs tend to be more robust and require less manual feature engineering.

Citations:

Yann LeCun, et al. (2015). Deep learning. Nature.

-Industrial IoT (IIoT) + Data-Driven Predictive Analytics

The use of sensors in manufacturing systems allows machines to continuously collect data such as temperature, vibration, and torque. This data can be analyzed using machine learning models to monitor machine health and predict failures before they occur. This approach is widely used in predictive maintenance and helps reduce downtime and improve efficiency.

Citations:

Jay Lee, et al. (2015). A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems. ScienceDirect.

-Unsupervised Learning and Anomaly Detection

In many manufacturing settings, labeled data is limited or unavailable. Because of this, unsupervised learning methods such as clustering and dimensionality reduction are commonly used. These techniques can help identify patterns in the data and detect unusual behavior that may indicate faults or abnormal operating conditions.

Citations:

Varun Chandola, et al. (2009). Anomaly Detection: A Survey. ResearchGate.

2.)
Since the collected data is unlabeled, an unsupervised clustering technique must be used.

The method should be is DBSCAN, and the advantage is that it does not require specifying the number of clusters beforehand and can effectively identify noise and outliers. In this problem, chatter events are rare and sporadic, making them similar to anomalies in the dataset. DBSCAN can isolate these abnormal points as noise, which makes it particularly suitable for detecting chatter.

3.)
a: Nonlinear Feature Relationships: Mutual Information

Mutual Information (MI) measures the dependency between two variables and can capture nonlinear relationships.

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \left(\frac{p(x,y)}{p(x)p(y)} \right)$$

where:

- p(x, y) is the joint probability distribution of X and Y
- p(x) and p(y) are the marginal probability distributions

MI is useful for feature selection because it does not assume linear relationships and can detect complex dependencies between features and the response variable.

b: Continuous Features with Categorical Output: LDA

Linear Discriminant Analysis (LDA) is used to maximize class separability for classification problems.

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

where:

- S_B is the between-class scatter matrix
- S_W is the within-class scatter matrix

These matrices are defined as:

$$S_B = \sum_{i=1}^{c} N_i(\mu_i - \mu)(\mu_i - \mu)^T$$

$$S_W = \sum_{i=1}^{c} \sum_{x \in D_i} (x - \mu_i)(x - \mu_i)^T$$

where:

- c is the number of classes
- N_i is the number of samples in class
- μ_i is the mean of class
- μ is the overall mean
- D_i is the set of samples in class

LDA selects features by maximizing the distance between class means while minimizing variance within each class.

4.)

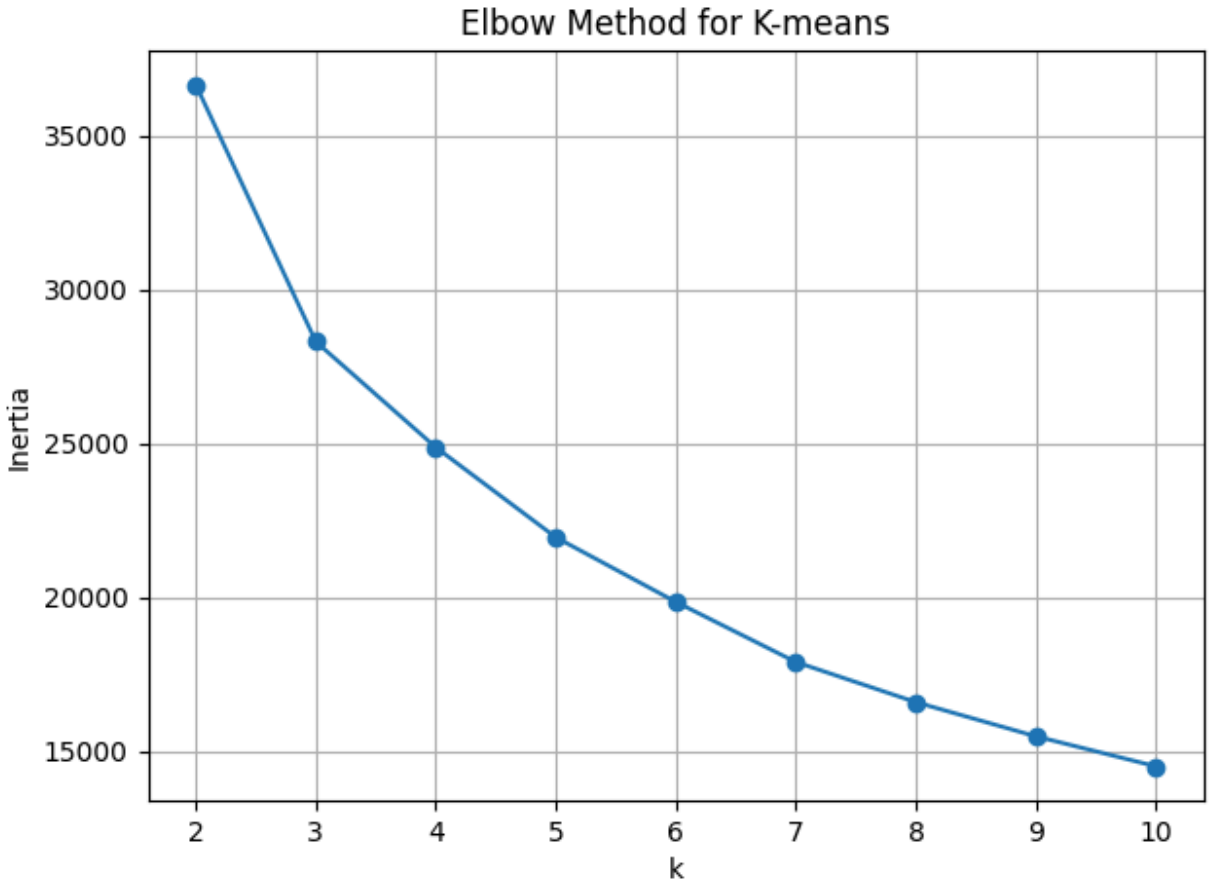
a:

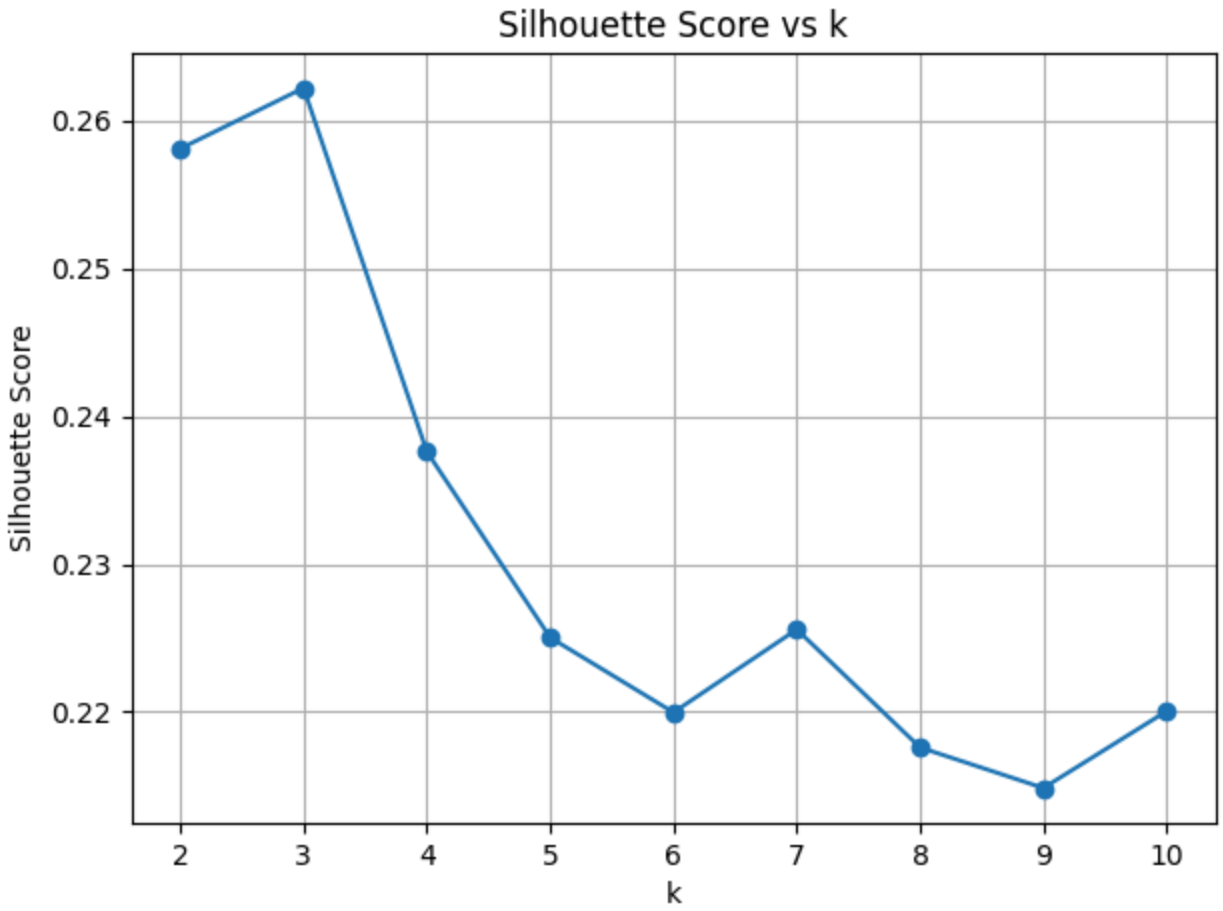
Code:

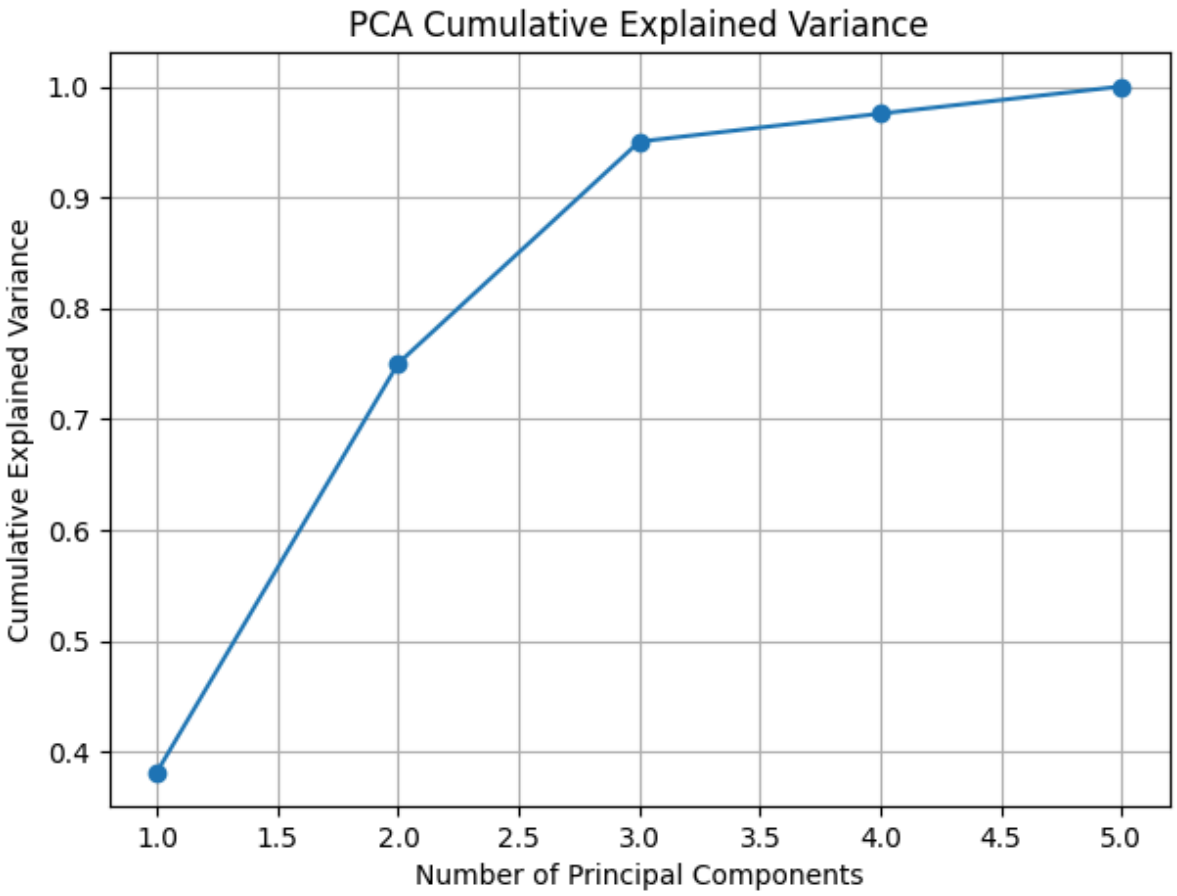
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score

# Load dataset
df = pd.read_csv("ai4i2020.csv")

# Use only continuous variables
X = df[
    [
        "Air temperature [K]",
        "Process temperature [K]",
        "Rotational speed [rpm]",
        "Torque [Nm]",
        "Tool wear [min]"
    ]
].copy()

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ----------------------------
# K-MEANS FOR MULTIPLE k VALUES
# ----------------------------
k_values = range(2, 11)
inertias = []
sil_scores = []

for k in k_values:
```

```
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)

    inertias.append(kmeans.inertia_)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)

    print(f"k={k}, inertia={kmeans.inertia_:.2f}, silhouette={sil:.4f}")

# Plot elbow curve
plt.figure(figsize=(7,5))
plt.plot(list(k_values), inertias, marker='o')
plt.xlabel("k")
plt.ylabel("Inertia")
plt.title("Elbow Method for K-means")
plt.grid(True)
plt.show()

# Plot silhouette scores
plt.figure(figsize=(7,5))
plt.plot(list(k_values), sil_scores, marker='o')
plt.xlabel("k")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs k")
plt.grid(True)
plt.show()

# ----------------------------
# CHECK LABEL AGREEMENT
# ----------------------------
for k in [3, 4, 5]:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X_scaled)

    ari_type = adjusted_rand_score(df["Type"], cluster_labels)
    nmi_type = normalized_mutual_info_score(df["Type"], cluster_labels)

    ari_fail = adjusted_rand_score(df["Machine failure"], cluster_labels)
    nmi_fail = normalized_mutual_info_score(df["Machine failure"],
cluster_labels)
```

```
    print(f"\nResults for k={k}")
    print(f"Type label -> ARI: {ari_type:.6f}, NMI: {nmi_type:.6f}")
    print(f"Machine failure label -> ARI: {ari_fail:.6f}, NMI:
{nmi_fail:.6f}")

# ----------------------------
# PCA
# ----------------------------
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

explained = pca.explained_variance_ratio_
cum_explained = np.cumsum(explained)

print("\nExplained variance ratio:")
for i, val in enumerate(explained, start=1):
    print(f"PC{i}: {val:.4f}")

print("\nCumulative explained variance:")
for i, val in enumerate(cum_explained, start=1):
    print(f"PC1 to PC{i}: {val:.4f}")

# Plot cumulative explained variance
plt.figure(figsize=(7,5))
plt.plot(range(1, len(cum_explained)+1), cum_explained, marker='o')
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA Cumulative Explained Variance")
plt.grid(True)
plt.show()
```


Elbow Method for K-means
Inertia
35000
30000
25000
20000
15000
2
3
4
5
6
7
8
9
10
k


Silhouette Score vs k
Silhouette Score
0.26
0.25
0.24
0.23
0.22
2
3
4
5
6
7
8
9
10
k



OutPut:

k=2, inertia=36654.19, silhouette=0.2581
k=3, inertia=28305.69, silhouette=0.2622
k=4, inertia=24870.55, silhouette=0.2376
k=5, inertia=21946.72, silhouette=0.2250
k=6, inertia=19848.98, silhouette=0.2200
k=7, inertia=17896.03, silhouette=0.2256
k=8, inertia=16590.97, silhouette=0.2176
k=9, inertia=15481.94, silhouette=0.2149
k=10, inertia=14507.21, silhouette=0.2201

Results for k=3
Type label -> ARI: -0.001088, NMI: 0.000416
Machine failure label -> ARI: -0.001876, NMI: 0.005454

Results for k=4
Type label -> ARI: -0.001921, NMI: 0.000402

Machine failure label -> ARI: 0.002636, NMI: 0.004162

Results for k=5
Type label -> ARI: -0.000455, NMI: 0.000301
Machine failure label -> ARI: 0.000061, NMI: 0.007128

Explained variance ratio:
PC1: 0.3821
PC2: 0.3682
PC3: 0.1999
PC4: 0.0253
PC5: 0.0245

Cumulative explained variance:
PC1 to PC1: 0.3821
PC1 to PC2: 0.7503
PC1 to PC3: 0.9502
PC1 to PC4: 0.9755
PC1 to PC5: 1.0000

Five continuous variables were used:

- Air temperature [K]
- Process temperature [K]
- Rotational speed [rpm]
- Torque [Nm]
- Tool wear [min]

The inertia decreases steadily as k increases, which is expected. However, the silhouette score reaches its maximum at k=3, indicating the best clustering quality at this value.

To evaluate whether clusters correspond to known labels, the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) were computed.

```
Results for k=3
Type label -> ARI: -0.001088, NMI: 0.000416
Machine failure label -> ARI: -0.001876, NMI: 0.005454

Results for k=4
Type label -> ARI: -0.001921, NMI: 0.000402
Machine failure label -> ARI: 0.002636, NMI: 0.004162

Results for k=5
Type label -> ARI: -0.000455, NMI: 0.000301
Machine failure label -> ARI: 0.000061, NMI: 0.007128
```

These values are all approximately zero, indicating no meaningful agreement between the clusters and the existing labels.

This result suggests that the clustering structure is driven by underlying process-variable relationships rather than the predefined product type or failure conditions.

b:
The optimal number of clusters is k=3.

This conclusion is supported by:
-Silhouette Score: The highest value (0.2622) occurs at k=3, indicating the best-defined clusters.
-Elbow Method: The inertia shows a significant drop from k=2 to k=3, after which the improvement becomes more gradual.

Thus, k=3 provides the best trade-off between model simplicity and clustering performance.

c:
PCA is meaningful for this dataset because a small number of principal components (2–3) capture most of the variance, indicating redundancy among the original features and enabling dimensionality reduction without significant loss of information.